

[next](#) | [previous](#) | [forward](#) | [backward](#) | [up](#) | [top](#) | [index](#) | [toc](#) | [home](#)

[Macaulay2](#) > [The Macaulay2 language](#) > [printing to the screen](#)

printing to the screen

Use the operator `<<` to print something to the screen.

```
i1 : << 2^100
1267650600228229401496703205376
o1 = stdio
o1 : File
```

Notice that the value returned is the standard input output file [stdio](#). We can also use `<<` as a binary operator to print explicitly to this file, and the output will appear on the screen.

```
i2 : stdio << 2^100
1267650600228229401496703205376
o2 = stdio
o2 : File
```

The associativity of the operator `<<` during parsing is arranged so that the following code will work in a useful way.

```
i3 : << "The answer is " << 2^100 << ".";
The answer is 1267650600228229401496703205376.
```

Printing works fine with nets, too.

```
i4 : << "The answer is " << "aaa" || "bb" || "c" << ".";
The answer is aaa.
          bb
          c
```

In the presence of nets, which are used whenever you print something that is formatted two-dimensionally, the only correct way to terminate a line is with [endl](#), even though a string called [newline](#) is available to you.

```
i5 : R = ZZ[x,y];
i6 : f = (x+y+1)^2; g = (x-y+1)^2
      2      2
o7 = x  - 2x*y + y  + 2x - 2y + 1
o7 : R
i8 : << "f = " << f << endl << "g = " << g << endl;
      2      2
f = x  + 2x*y + y  + 2x + 2y + 1
      2      2
g = x  - 2x*y + y  + 2x - 2y + 1
```

Output is saved in a buffer until the end of the line is encountered. If nets are not involved, you may use [flush](#) to force the output to appear on the screen before the end of the line is encountered. This may be useful in displaying some tiny indication of computational progress to the user.

```
i9 : scan(0 .. 20, i -> << "." << flush)
.....
```

If long lines get displayed too slowly, such as in emacs, then the user may choose to put a line such as `truncateOutput 100` into an [initialization file](#). Time is still spent creating the wide output which is eventually truncated.

```
i10 : truncateOutput 50
```

i11 : 41!

o11 = 3345252661316380710817006205344075166515200000000

i12 : 42!

o12 = 1405006117752879898543142606244511569936384000 ...

i13 : 43!

o13 = 6041526306337383563735513206851399750726451200 ...

i14 : truncateOutput infinity

o14 = identity

o14 : CompiledFunction

i15 : 43!

o15 = 6041526306337383563735513206851399750726451200000000